Inertial odometry with a livox-MID360 lidar

Project Number 9

Andrea Cusumano

Giuseppe Terranova

Daniela Previtera

Università di Catania

LiDAR-inertial SLAM system

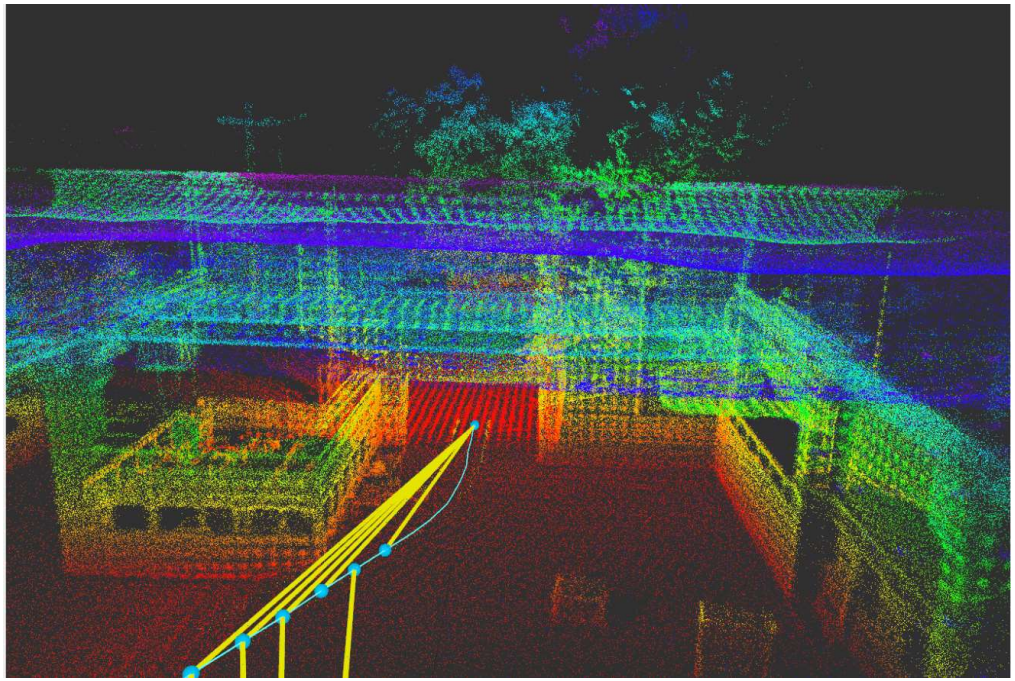
In this project, the lio_sam_mid360[1] repository was used to implement a **LiDAR-inertial SLAM system** based on the integration of data from the Livox MID-360 and an IMU.

This repository provides the ROS 2 package needed to process the point cloud generated by the LiDAR, ***together*** with the inertial measurements provided by the IMU.

The LiDAR is used to acquire **the geometric structure** of the surrounding environment, while the IMU helps estimate the **motion** of the robot over time.

By combining these two sources of information, the system is able to estimate the robot pose during movement and incrementally build a three-dimensional map of the explored area.

The resulting trajectory and 3D point cloud map can then be visualized in RViz and saved for further analysis or future navigation applications

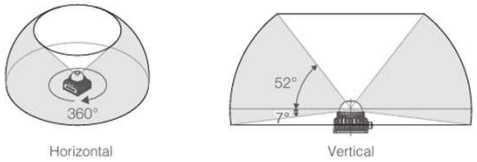


[1] HTTPS://GITHUB.COM/RAJVISHNU07/LIO_SAM_MID360?TAB=README-OV-FILE#README

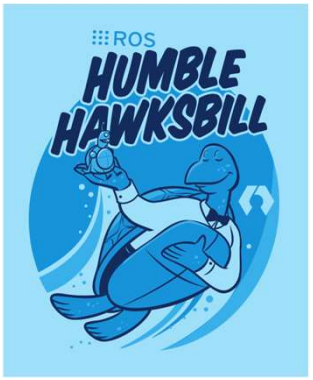
Hardware and software

Livox Mid-360

Field Of View



FIELD OF VISION	POWER SUPPLY	CONNECTION
Horizontal: 360°, Vertical: -7°~52°	9~27 V DC Optimal 12V	Ethernet cable



Ros2 Humble

ubuntu 22.04 LTS

WIRINGS:

The LiDAR is powered by a bench power supply set to 12V, which is its optimal operating voltage.
Communication with the sensor is established via an Ethernet cable

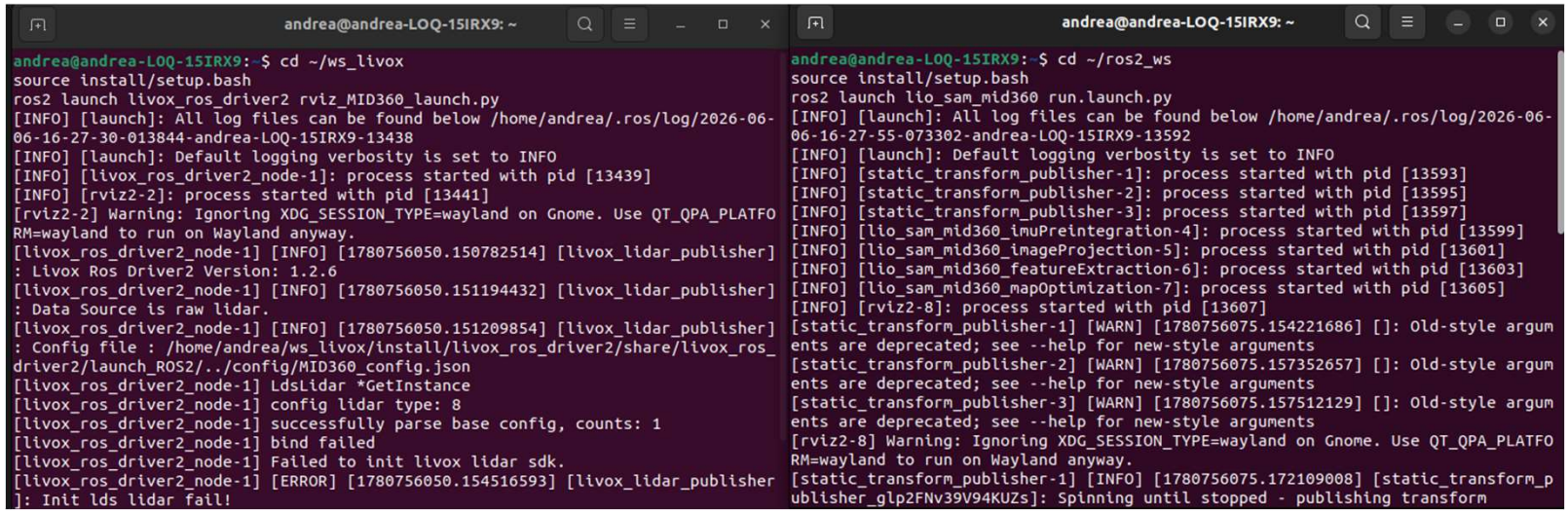
CONNECTION STEPS:

1) Identify the logic name of the PC's ethernet port with the command "*lshw -class network*"
2) Set the static IP of the ethernet connection with the command: sudo ifconfig “logic_name” 192.168.1.50
3) Connect the ethernet cable

To check if the device is connected, the Livox software can be launched

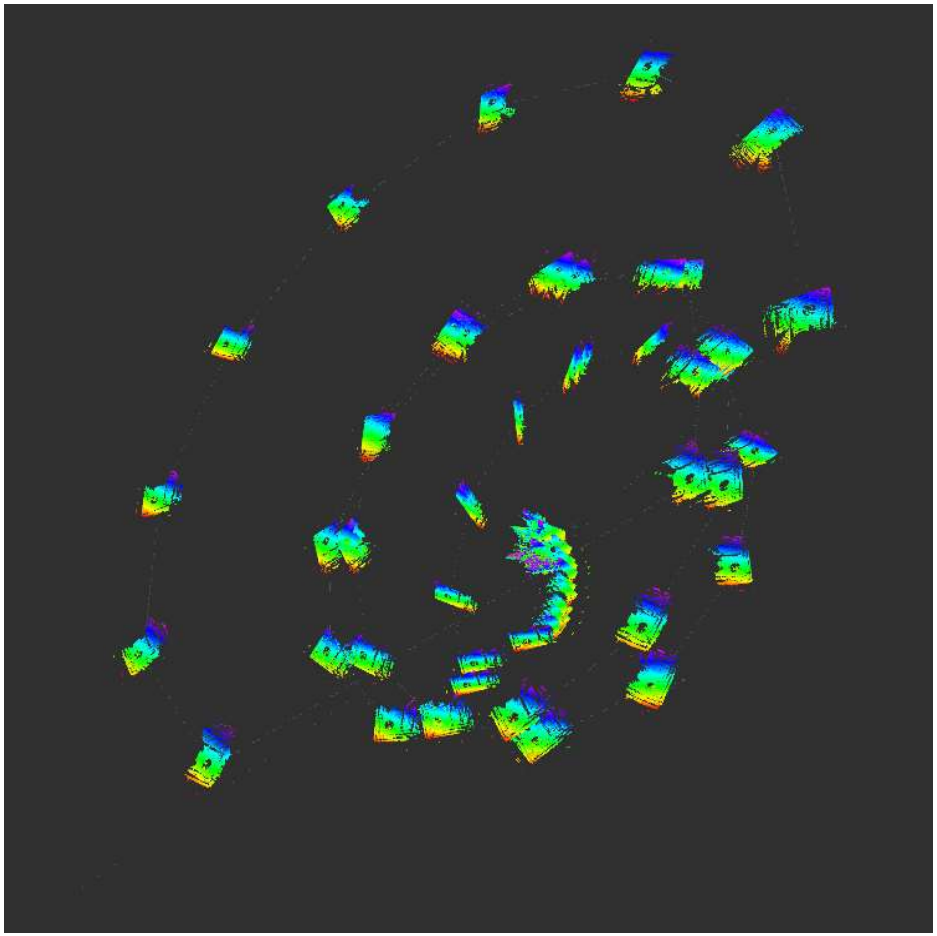
Launching the repository

To execute the repository, we configured two workspaces: one for the `livox_driver`[2] and one for the repository files[1]. Running the system requires opening the driver and the launcher in separate terminals.



The first one opens the RealTime acquisition, whereas the second one has the main program, where the map is generated

[2] HTTPS://GITHUB.COM/LIVOX-SDK/LIVOX_ROS_DRIVER2



RVIZ2 – spiral of doom

Once the setup was complete, we immediately faced a critical issue with the internal IMU configuration. As highlighted in the left image, a severe IMU error heavily distorted our position calculations. Instead of mapping correctly, the system continuously updated at coordinates **progressively further away in space**, forcing the entire map to drift outward along an ever-expanding spiral.

This error is due to the IMU's calibration.

IMU CALIBRATION

```
# IMU Settings
imuType: 0

imuAccNoise: 9.856e-02
imuGyrNoise: 3.499e-03
imuAccBiasN: 0.19
imuGyrBiasN: 0.04
imuGravity: 9.8035
imuRPYWeight: 0.00
```

To replace the generic default IMU noise parameters, we **empirically** estimated the **specific** noise characteristics of our sensor.

We collected five minutes of linear acceleration and angular velocity data via a rosbag on the /livox/imu topic.

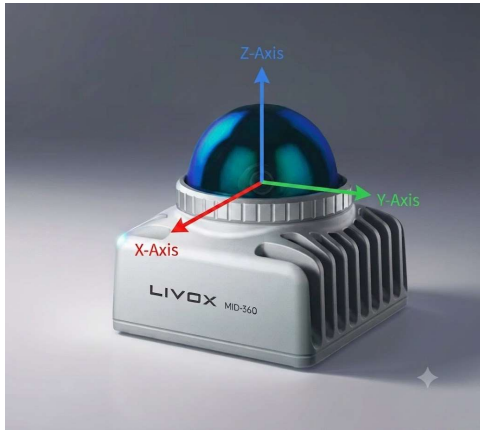
After calculating the mean and standard deviation using a Python script, we updated the params.yaml configuration file.

This custom calibration effectively minimized sensor drift, enhancing the overall accuracy and stability of the generated map.

FIRST MOBILE SETUP

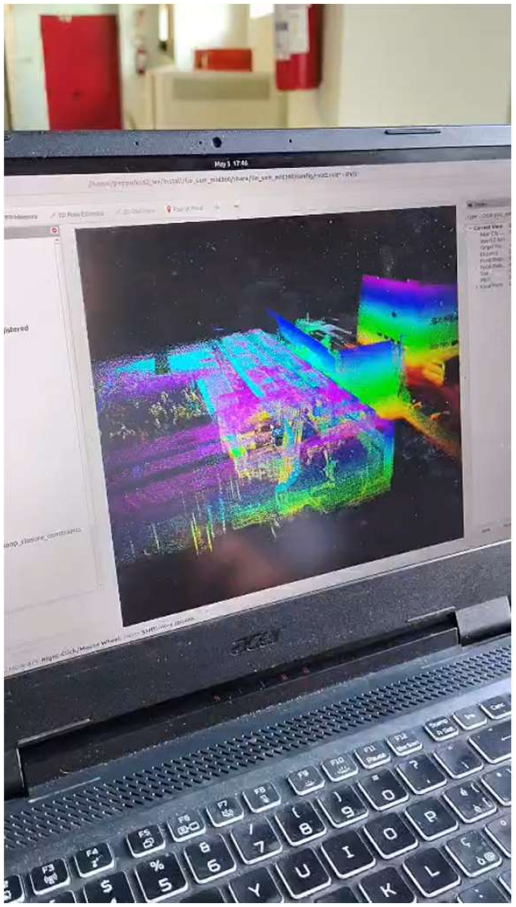
As a first experimental test, the Livox sensor was mounted on a mobile table and placed in the front area of the laboratory.

Then, the device was tested in movement, both indoor and outdoor.



The lidar was placed such that our forward direction was the same as its x-axis.

CALIBRATION AND FIRST MOBILE RESULTS



Even though the floor surface was not smooth and many vibrations occurred during motion, the 'spiral of doom' drifting did not occur.
As a result, the map generation process went smoothly, and both indoor and outdoor areas were successfully mapped onto a point cloud.
The video on the left presents these results, where we can observe not only the mapped building but also the route followed by our mobile robot (moving table).

Since our configuration is now complete, we can proceed to the application on the Robovolc.

Connection with Robovolc

A ***Robo***t for ***Volc***ano Exploration

- **Robovolc** acts as the **mobile robotic platform** that allows the sensor to be moved through the environment during the acquisition process.
- To connect to **Robovolc**, we first need to connect to its internal network, called `robovolc-AP`, which is generated by the robot's own access point.
- On board the robot, there is a **NUC** that connects to another access point through a **USB Wi-Fi dongle**.
- This connection is used to provide Internet access. The NUC also runs the DHCP service, which automatically assigns IP addresses to the devices connected to the robot's internal network.

In this way, the Internet connection is **shared from the NUC to all the devices on the network**, allowing them to communicate with each other and access external **services** when needed.

SERVER DHCP

Linux computer as router and DHCP server

- Using ***`iptables`,*** NAT is enabled. This allows the devices connected to the local network to access the Internet using the server's address.

sudo nano /etc/iptables/rules.v4

✓ Additional rules are added to allow traffic forwarding from the LAN to the Internet-facing interface and to allow response packets to return correctly to the clients.

To avoid manually configuring each device on the network, a DHCP server is also installed → ***sudo nano /etc/dhcp/dhcpd.conf***

✓ The DHCP server automatically assigns an IP address to the hosts in a selected range.

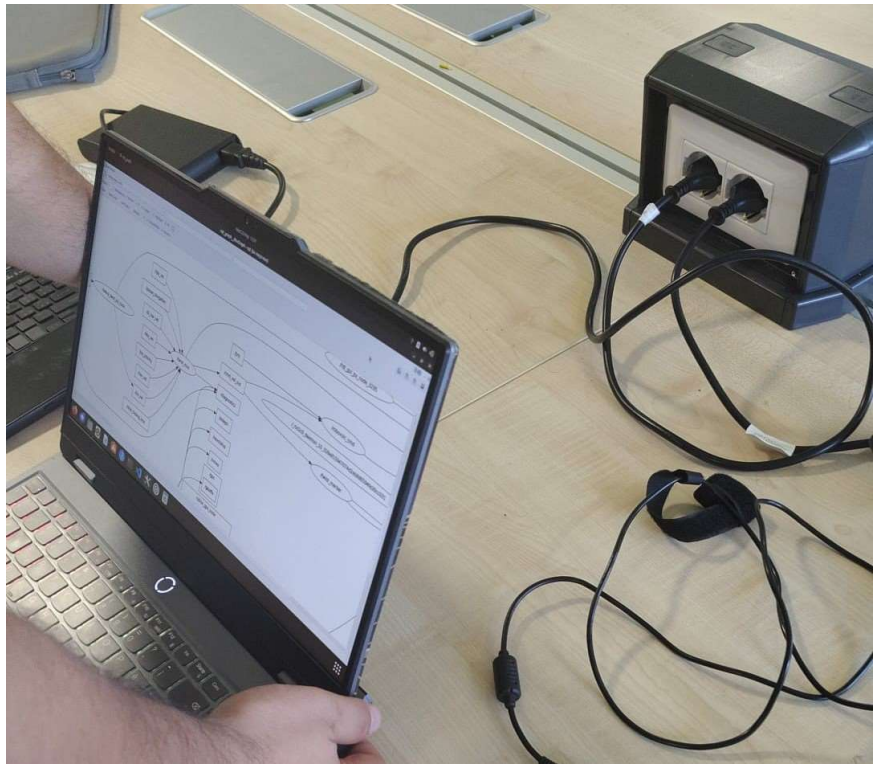
✓ The configuration also defines the lease **duration for the assigned IP addresses**.

✓ Finally, in the `/etc/default/isc-dhcp-server` file, the interfaces on which the DHCP server should listen are specified.

After modifying the configuration files, the DHCP service is restarted to apply the new settings.

Command to login

ssh -Y *admin@ipaddress*



rqt_graph provides a GUI plugin for visualizing the ROS computation graph.

Velocity Control

ros2 run teleop_twist_keyboard teleop_twist_keyboard --ros-args --remap cmd_vel:=/key_vel

```
For Holonomic mode (strafing), hold down the shift key:
---------------------------------------------------
   U    I    O
   J    K    L
   M    <    >

t : up (+z)
b : down (-z)

anything else : stop

q/z : increase/decrease max speeds by 10%
w/x : increase/decrease only linear speed by 10%
e/c : increase/decrease only angular speed by 10%

CTRL-C to quit

currently:      speed 0.17259582784950006       turn 0.1593705494741008
currently:      speed 0.17259582784950006       turn 0.1753076044215109
```

This command starts the teleop_twist_keyboard ROS 2 node, which allows the robot to be **controlled manually using the keyboard.**

The node converts keyboard inputs into velocity commands of type geometry_msgs/msg/Twist.

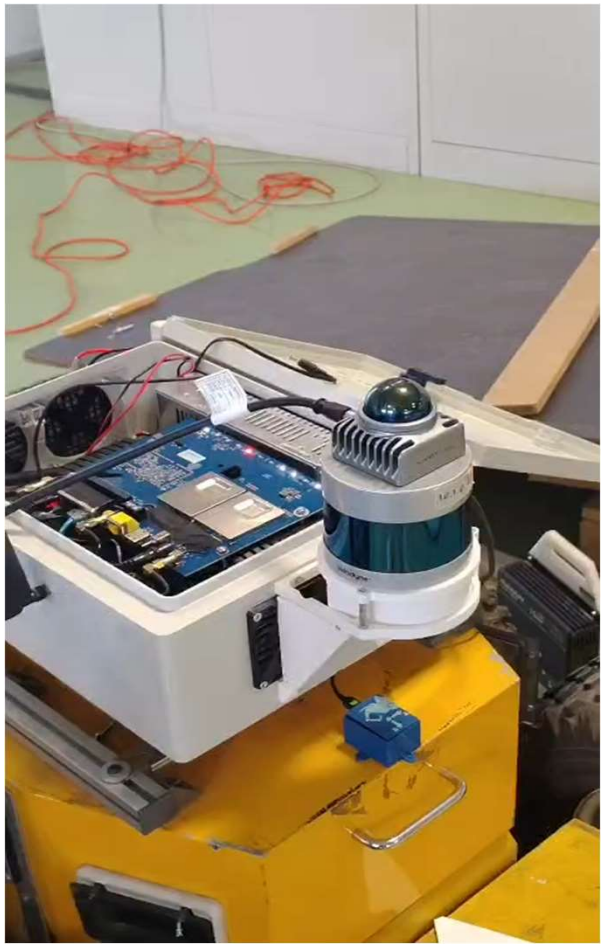
Normally, the commands are published on the /cmd_vel topic

In ROS 2, topics are communication channels used by nodes to exchange messages, in this case a remapping is used so that they're published on /key_vel instead on /cmd_vel.

This is useful when the keyboard velocity commands need to be **separated** from other velocity sources, such as autonomous navigation or joystick control

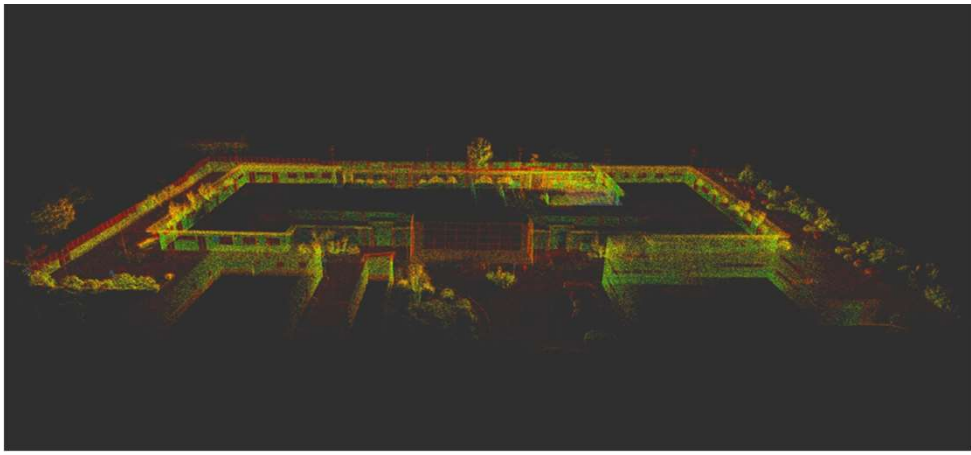
robovolc.service manages the core software needed for sensor drivers and low-level robot control, including joystick control.

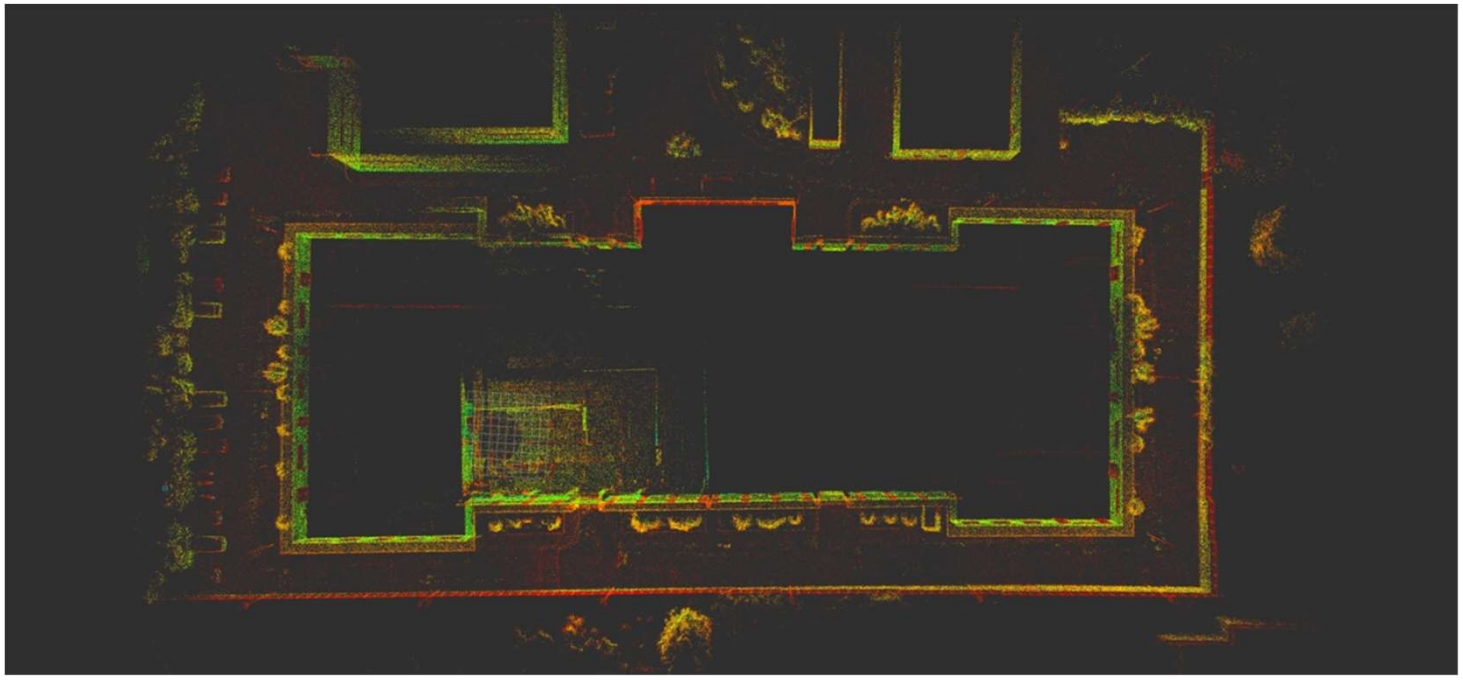
Final Setup



After mounting the LiDAR on the robot and completing the overall configuration of the reference frames, particular attention was paid to firmly securing the sensor and ensuring its correct position and orientation.
We finally moved outdoors to start mapping the perimeter of the Polo Tecnologico.

Map Saving and Reloading

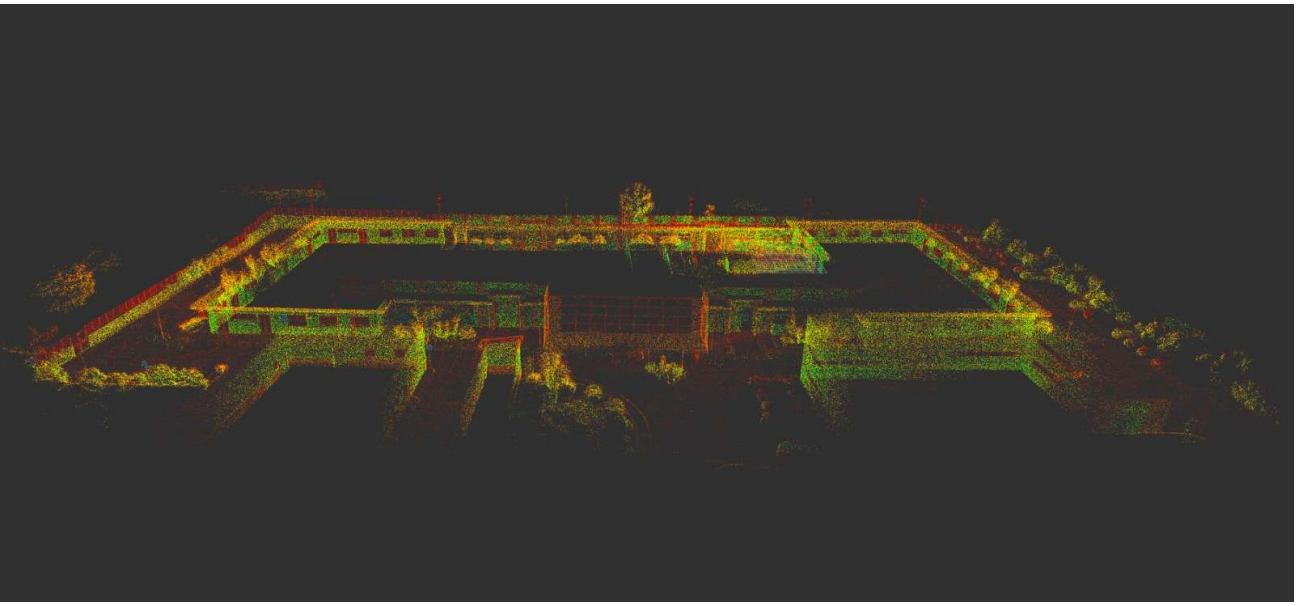


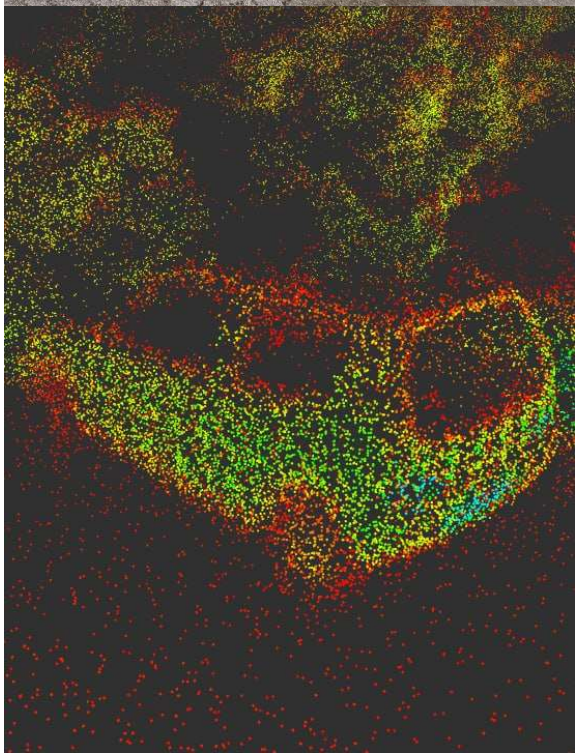


After the map generation process, the map was saved using the ***/lio_sam/save_map*** service, specifying the desired resolution and destination folder

To load the map, the saved PCD file was published as a ROS 2 PointCloud2 topic using the pcl_ros package.

The point cloud was then visualized in RViz2 through a dedicated configuration file optimized for map visualization.





The FIAT 600 mapping shows the high level of detail captured in the generated point cloud. (Andrea's car)

Final Results

At the beginning, we experienced significant **drifting issues** due to vibrations affecting the LiDAR mounted on the robot.

These vibrations caused **huge rotations** of the sensor during motion, which compromised the initial calibration and reduced the accuracy of the mapping process.

After firmly securing the LiDAR and **lowering the robot's speed**, we were able to achieve a highly accurate mapping of the entire area.

THANKS FOR YOUR ATTENTION!

Polo’s Bunny